

FACULDADE COTEMIG

FUNDAMENTOS TEÓRICOS DA COMPUTAÇÃO

**RELATÓRIO TÉCNICO: SIMULADOR DE MÁQUINAS
ABSTRATAS**

Membros da Equipe:

Alex Paulo Vianna Da Costa Candido e Silva

Carlos Henrique Queiroz Teixeira Ramos

Bernardo Medeiros Mendes

Belo Horizonte / MG

Junho de 2026

1 Introdução

O presente relatório documenta o desenvolvimento de um simulador computacional de máquinas abstratas, requisito para a avaliação final da disciplina de Fundamentos Teóricos da Computação. O objetivo central deste trabalho é transpor modelos matemáticos teóricos — Autômatos Finitos Determinísticos (AFD), Autômatos de Pilha (AP) e Máquinas de Turing (MT) — para algoritmos executáveis em linguagem C#.

O estudo de linguagens formais e autômatos fornece a base matemática para a compreensão do que pode ou não ser computado. Através deste projeto, buscou-se validar conceitos como determinismo, manipulação de memória restrita (pilhas) e memória irrestrita (fita infinita). O sistema foi arquitetado de forma modular, com validações baseadas nas tuplas formais que definem cada máquina, permitindo não apenas o reconhecimento de linguagens específicas, mas também a resolução de problemas de decisão e funções matemáticas (como a adição unária).

2 Fundamentação Teórica

2.1 Autômato Finito Determinístico (AFD)

Um AFD é um modelo matemático restrito que reconhece Linguagens Regulares, formalmente definido por uma 5-tupla $M = (Q, \Sigma, \delta, q_0, F)$, onde: Q é um conjunto finito de estados; Σ é o alfabeto de entrada; $\delta : Q \times \Sigma \rightarrow Q$ é a função de transição; $q_0 \in Q$ é o estado inicial; e $F \subseteq Q$ é o conjunto de estados finais. **Exemplo de Cálculo (L_1 - Termina em ab):** Ao processar a palavra $w = aab$, a máquina parte de q_0 . Lê ' a ' $\rightarrow q_1$. Lê ' a ' $\rightarrow q_1$. Lê ' b ' $\rightarrow q_2$. Como $q_2 \in F$, a palavra é aceita.

2.2 Autômato de Pilha (AP)

Um AP reconhece Linguagens Livres de Contexto, possuindo uma memória auxiliar LIFO. É definido pela 7-tupla $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$. No nosso projeto, a aceitação ocorre *exclusivamente por pilha vazia* (logo, $F = \emptyset$). **Exemplo de Cálculo ($L_2 = a^n b^n$):** Para $w = aabb$, a máquina lê ' a ' e empilha ' A ' duas vezes. Através de um λ -movimento, adivinha a transição para os ' b 's e, a cada ' b ' lido, desempilha um ' A '. Ao final da cadeia, a pilha contém apenas Z_0 , que é desempilhado, aceitando a palavra.

2.3 Máquina de Turing (MT)

A MT é o modelo mais poderoso, capaz de reconhecer Linguagens Recursivamente Enumeráveis, possuindo uma fita infinita. Definida por $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$. **Exemplo de Cálculo ($L_4 = a^n b^n c^n$):** Ao processar abc , a MT lê ' a ', escreve ' X ' e move para a direita (R) até achar ' b ', escreve ' Y ' e move (R) até achar ' c ', escrevendo ' Z '. Em seguida, move para a esquerda

(L) para reiniciar a marcação. Se encontrar apenas brancos (B) no final do processo, aceita a palavra.

3 Descrição da Implementação

O projeto foi desenvolvido em .NET (C#), priorizando o mapeamento direto da teoria para estruturas de dados modernas.

3.1 Implementação do AFD

O AFD foi estruturado utilizando a classe `HashSet<T>` do C# para representar os conjuntos finitos matemáticos (Q, Σ, F) , garantindo unicidade dos elementos e complexidade de busca $O(1)$. A função de transição δ foi modelada como um `Dictionary<(string, char), string>`, mapeando perfeitamente a relação estado-símbolo para um novo estado. Além das validações no código (linguagem L_1), foi implementado o Desafio Obrigatório que carrega um AFD genérico via arquivo JSON (`System.Text.Json`).

3.2 Implementação do Autômato de Pilha

Para o AP, a fita auxiliar foi implementada estritamente com a classe nativa `Stack<char>`. Para suportar o não-determinismo exigido pelo desafio dos Palíndromos (L_3), a função de transição precisou retornar uma `List` de caminhos possíveis, associada a uma simulação recursiva de estados.

3.3 Implementação da Máquina de Turing

A maior inovação do projeto ocorreu na implementação da Fita Infinita da Máquina de Turing. Optou-se por utilizar um `Dictionary<int, char>`, onde a chave é o índice da fita (que pode assumir valores negativos infinitos) e o valor é o símbolo. Isso evita os problemas de redimensionamento de memória que `Arrays` ou `Lists` apresentariam.

4 Discussão das Questões Reflexivas

Abaixo, apresentamos as considerações teóricas e práticas formuladas durante a concepção de cada máquina abstrata, com respostas extensas para o embasamento rigoroso do projeto.

4.1 Parte 1: Autômato Finito Determinístico (AFD)

1. Por que a linguagem L_1 é regular? Apresente a expressão regular equivalente.

Resposta: A linguagem L_1 é classificada como regular porque, por definição fundamental da

Teoria da Computação, toda linguagem que pode ser inteiramente reconhecida e decidida por um Autômato Finito (seja determinístico ou não) com memória restrita a estados é uma linguagem regular. Ao construirmos de forma exata um AFD que avalia e aceita estritamente as cadeias que encerram com o sufixo "ab", nós provamos na prática a sua regularidade. Matematicamente, o teorema de equivalência de Kleene dita que linguagens regulares também podem ser sintetizadas por meio de Expressões Regulares (ER). Para descrever a linguagem L_1 , a expressão regular equivalente e minimizada é dada pela fórmula $r = (a + b)^*ab$. Essa modelagem algébrica determina que uma palavra válida pode iniciar com qualquer combinação finita de símbolos do alfabeto (representado pelo fecho de Kleene asterisco), mas seu encerramento deve ser imperativamente ditado pela concatenação do símbolo 'a' seguido imediatamente do símbolo 'b'.

2. Descreva formalmente a 5-tupla do AFD construído, justificando a escolha de cada estado.

Resposta: O Autômato Finito Determinístico formulado pode ser matematicamente traduzido pela 5-tupla $M = (Q, \Sigma, \delta, q_0, F)$. Os componentes mapeados em nosso código C# definem o conjunto finito de estados como $Q = \{q_0, q_1, q_2\}$, o alfabeto de entrada como $\Sigma = \{a, b\}$, o estado inicial como $q_0 = q_0$ e o conjunto de aceitação contendo apenas $F = \{q_2\}$. A arquitetura dos estados foi arquitetada para reconhecer um padrão de sufixo: o estado q_0 figura como o estágio inicial ou de repouso, significando que a máquina ainda não leu nenhuma parte da sequência esperada (o 'a'). O estado q_1 é adotado quando o autômato identifica a letra 'a', assinalando que metade do reconhecimento ocorreu e aguarda-se o engate da próxima transição. O estado q_2 é justificado como a confirmação final da leitura da letra 'b', carimbando o estado de aceitação. Se o autômato se encontra em q_2 e lê caracteres extras, ele quebra o padrão final e recua imediatamente para q_1 ou q_0 .

3. O que aconteceria se a função δ não fosse total? Como você tratou entradas inválidas (símbolos fora do alfabeto)?

Resposta: Do ponto de vista puramente teórico, se a função de transição δ de um AFD não abrangesse a totalidade do plano cartesiano ($Q \times \Sigma$), a máquina ostentaria lacunas severas em seu diagrama e paralisaria caso encontrasse um par de estado-símbolo indefinido, falhando em fornecer uma computação determinística completa. Na nossa codificação, contornamos essa não-totalidade simulando um "estado de poço" implícito por intermédio da biblioteca nativa da linguagem: ao acionar o método TryGetValue do Dicionário, uma falha de mapeamento resulta em retorno falso, o que causa o colapso proposital e a rejeição instantânea da cadeia processada. Além desse arranjo para transições ausentes, arquitetamos um mecanismo contra a invasão de entradas inválidas ou alienígenas; uma instrução condicional inicial valida se o caractere sob análise habita o escopo numérico do conjunto Σ . Caso um símbolo fora do alfabeto tente infiltrar-se no processamento, o cômputo é abortado preventivamente para evitar anomalias estruturais.

4.2 Parte 2: Autômato de Pilha (AP)

1. Por que L_2 não pode ser reconhecida por um AFD? Demonstre usando o Lema do Bombeamento para linguagens regulares.

Resposta: A impossibilidade de a linguagem $L_2 = \{a^n b^n \mid n \geq 1\}$ ser computada em um AFD reside em uma incapacidade arquitetônica estrutural: os Autômatos Finitos são desprovidos de dispositivos de memória volumétrica auxiliar. Devido a isso, um autômato desse porte é incapaz de arquivar contagens elásticas e emparelhar as quantidades de duas naturezas textuais em um universo ilimitado de possibilidades. A validação absoluta dessa incapacidade baseia-se no teorema do Lema do Bombeamento. Suponhamos, buscando uma contradição flagrante, que L_2 seja regular e ostente um patamar constante de bombeamento p . Considerando a escolha arbitrária da cadeia longa $w = a^p b^p$, realizamos a clássica segmentação xyz . O próprio Lema impõe as restrições formais $|xy| \leq p$ e $|y| > 0$, significando que o agrupamento bombeável 'y' compõe-se estritamente de letras 'a'. Ao aplicarmos o bombeamento (y^i , no caso $i = 2$), estaríamos apenas injetando repetições infinitas da variável 'a', rompendo inteiramente o equilíbrio com a parte invariável dos 'b's e fabricando uma palavra errônea. Diante da contradição formal, solidifica-se que L_2 não é regular.

2. A aceitação por pilha vazia e por estado final são equivalentes em poder de reconhecimento? Demonstre conceitualmente.

Resposta: A afirmativa é plenamente correta; os paradigmas metodológicos de aceitação mediante o esgotamento por pilha vazia ou a admissão por atingimento de estado final ostentam uma equivalência exata na amplitude do reconhecimento das linguagens Livres de Contexto. Qualquer sistema mecânico modelado seguindo a vertente do estado final é passível de conversão engenhosa para o método da pilha exaurida, e vice-versa. Por exemplo, para metamorfosear uma máquina que dependa da inspeção do estado final num arranjo que exija a pilha nula, desenhasse um conjunto suplementar de λ -movimentos atrelados a todos os antigos estados terminais. Esses caminhos ociosos migrarão o fluxo incondicionalmente para uma nova "estação desempilhadora" singular, na qual todas as sobras da memória são devoradas sucessivamente até o vácuo se consumir, carimbando assim a legitimação por pilha nula sem perda no rigor teórico. Esse isomorfismo providencial outorga aos desenvolvedores flexibilidade estrutural incalculável.

3. Explique o papel do símbolo Z_0 na sua implementação.

Resposta: O emprego imperativo do caractere estipulado como Z_0 consubstancia uma rede dupla de estabilização nos parâmetros arquitetônicos do nosso APD. Em escala matemática e abstrata, esse símbolo age estritamente como a demarcação basilar ou âncora fundacional da pilha LIFO (*Last-In, First-Out*), assinalando o abismo da memória transiente em que as camadas foram encerradas. Sob o prisma de execução do software codificado na plataforma .NET C#, manusear objetos da tipologia `Stack` para extrair valores por meio de consultas vazias (seja usando `Pop` ou `Peek`) resultaria instantaneamente num destrutivo evento `StackEmptyException`,

aniquilando a continuidade do rastreamento. Ao pré-alocarmos a base com o sinal de Z_0 , forramos o assoalho de processamento para evitar colapsos precoces por inspeção de vazio absoluto. Assim que a máquina esgota a palavra processada, a revelação exclusiva desse marcador assegura incontestavelmente a coerência e igualdade da paridade simétrica submetida a teste na cadeia, induzindo à aceitação por esvaziamento limpo final.

4.3 Parte 3: Máquina de Turing (MT)

1. Por que L_4 não pode ser reconhecida por um Autômato de Pilha? Qual propriedade da MT possibilita o seu reconhecimento?

Resposta: A linguagem interrelacionada $L_4 = \{a^n b^n c^n \mid n \geq 1\}$ figura no patamar das linguagens sensíveis ao contexto, as quais transcendem de maneira fatal as capacidades de armazenamento de restrição de profundidade LIFO das memórias utilizadas pelos Autômatos de Pilha usuais. A operação que insere as letras 'a' na pilha resolve com maestria o pareamento correspondente à variável 'b', exaurindo o empilhamento para testar a equivalência da primeira metade da string formatada. O desastre impõe-se subsequentemente, pois, no momento em que a quantificação do índice 'b' é encerrada, todo o histórico linear memorizado foi corrompido ou dissipado irremediavelmente; portanto, falta parâmetro de confronto empírico para assegurar a idêntica quantia de repetições com as letras 'c'. As Máquinas de Turing neutralizam essas imperfeições e vencem o desafio porque portam a capacidade de registrar a escrita em uma fita infinita, combinada sinergicamente ao poder de alternar o cabeçote num esquema rotatório bidirecional para revisitar células em varreduras reiteradas, sem destruição acidental da infraestrutura passada armazenada.

2. Quantos estados foram necessários para a MT de L_4 ? Descreva a estratégia de “marcação” adotada.

Resposta: A nossa transcrição computacional rigorosa focada no desvendamento pragmático da linguagem matemática sensível ao contexto L_4 utilizou expressamente seis matrizes comportamentais denominadas pelos estados $\{q_0, q_1, q_2, q_3, q_4 \text{ e } q_{acc}\}$. Para alcançar a convergência correta, nossa simulação adotou a metodologia progressiva de rotulação de zigue-zague iterativo. Em uma passagem fundamental, o marcador situado em q_0 desbrava o caractere original de 'a' transformando-o inexoravelmente na variável gráfica de identificação 'X'. Imediatamente a seguir, atua o estágio prospectivo do q_1 , navegando linearmente à parte leste até esbarrar no correspondente virgem do primeiro registro da categoria 'b', convertendo esse em 'Y' no compasso transacional, para então delegar ao prospecto q_2 a repetição processual destinada unicamente ao mapeamento singular da demarcação 'c' final por um referencial cravado como 'Z'. Quando essa operação se conclui no limite meridional, entra no jogo o reverso de q_3 , fazendo a unidade física rebobinar exaustivamente saltando as relíquias lidas para recompor as marcações reiniciando de q_0 .

3. Um computador moderno é mais poderoso que uma Máquina de Turing? Justifique à luz da Tese de Church-Turing.

Resposta: No escopo rigorosamente teórico delineado para a fundação originária da computação abstrata, a assertiva encontra refutação frontal; nenhuma implementação mecânica hodierna transcende efetivamente os alcances e os limiares abarcados por intermédio das Máquinas de Turing. A famosa proposição consagrada mundialmente como Tese de Church-Turing manifesta, de forma assertiva e contundente, a hipótese elementar de que todo processo empírico factível por meio de rotinas dedutíveis lógicas ou de sistematizações não ambíguas restringe-se precisamente ao potencial calculista enraizado nessa abstração. Qualquer PC ultramoderno que envolva processamento quântico, clusters na nuvem distribuída e processadores lógicos de hiper escalabilidade atuará invariavelmente como apenas manifestações mecânicas do construto formalizado do matemático inglês. Na essência teórica e abstrata paradoxal do conceito, pode-se assegurar que os protótipos de Turing são hierarquicamente "invencíveis", pois a imaginação dispõe sobre eles faixas métricas sem limites (fitas infinitas), que nunca se igualam à castração real dos gigabytes finitos estocados nos pentes da RAM atuais.

5 Resultados e Testes

As saídas no terminal (Console) corroboram a corretude matemática das máquinas construídas. Os rastreamentos instantâneos das memórias (traces) mostraram-se exatos. Abaixo, os resultados processados na íntegra para os arquivos de entrada exigidos:

AFD (Parte 1)

Diagrama de Transições (Tabela)

Estado	a	b	
q0	q1	q0	
q1	q1	q2	
q2	q1	q0	

Processando entradas.txt para L1...

Cadeia: ab

Rastro: q0 -> q1 -> q2

Resultado: ACEITA

Cadeia: aab

Rastro: q0 -> q1 -> q1 -> q2

Resultado: ACEITA

```

Cadeia: bab
Rastro: q0 -> q0 -> q1 -> q2
Resultado: ACEITA
-----
Cadeia: ababab
Rastro: q0 -> q1 -> q2 -> q1 -> q2 -> q1 -> q2
Resultado: ACEITA
-----
Cadeia: ba
Rastro: q0 -> q0 -> q1
Resultado: REJEITA
-----
Cadeia: vazia
Rastro: q0
Resultado: REJEITA
-----
Cadeia: b
Rastro: q0 -> q0
Resultado: REJEITA
-----

```

```

Carregando afd.json
Máquina carregada com sucesso do arquivo JSON!
Diagrama de Transições (Tabela)
Estado      | 0          | 1
-----
q0           | q0         | q1         |
q1           | q1         | q0         |
-----

```

=== Simulador de Autômato de Pilha (Parte 2) ===

=== L2: $a^n b^n$ ($n \geq 1$) ===

```

Processando: ab
  Configuração: (q: q0, w: ab, Pilha: Z)
  Configuração: (q: q0, w: b, Pilha: AZ)
  Configuração: (q: q1, w: b, Pilha: AZ)
  Configuração: (q: q1, w: E, Pilha: Z)
  Configuração: (q: q2, w: E, Pilha: [Vazia])
Resultado: ACEITA
-----

```

```

Processando: aabb
  Configuração: (q: q0, w: aabb, Pilha: Z)
  Configuração: (q: q0, w: abb, Pilha: AZ)

```

Configuração: (q: q1, w: abb, Pilha: AZ)
 Configuração: (q: q0, w: bb, Pilha: AAZ)
 Configuração: (q: q1, w: bb, Pilha: AAZ)
 Configuração: (q: q1, w: b, Pilha: AZ)
 Configuração: (q: q1, w: E, Pilha: Z)
 Configuração: (q: q2, w: E, Pilha: [Vazia])
 Resultado: ACEITA

Processando: aaabbb

Configuração: (q: q0, w: aaabbb, Pilha: Z)
 Configuração: (q: q0, w: aabbb, Pilha: AZ)
 Configuração: (q: q1, w: aabbb, Pilha: AZ)
 Configuração: (q: q0, w: abbb, Pilha: AAZ)
 Configuração: (q: q1, w: abbb, Pilha: AAZ)
 Configuração: (q: q0, w: bbb, Pilha: AAAZ)
 Configuração: (q: q1, w: bbb, Pilha: AAAZ)
 Configuração: (q: q1, w: bb, Pilha: AAZ)
 Configuração: (q: q1, w: b, Pilha: AZ)
 Configuração: (q: q1, w: E, Pilha: Z)
 Configuração: (q: q2, w: E, Pilha: [Vazia])
 Resultado: ACEITA

Processando: aab

Configuração: (q: q0, w: aab, Pilha: Z)
 Configuração: (q: q0, w: ab, Pilha: AZ)
 Configuração: (q: q1, w: ab, Pilha: AZ)
 Configuração: (q: q0, w: b, Pilha: AAZ)
 Configuração: (q: q1, w: b, Pilha: AAZ)
 Configuração: (q: q1, w: E, Pilha: AZ)
 Resultado: REJEITA

Processando: abb

Configuração: (q: q0, w: abb, Pilha: Z)
 Configuração: (q: q0, w: bb, Pilha: AZ)
 Configuração: (q: q1, w: bb, Pilha: AZ)
 Configuração: (q: q1, w: b, Pilha: Z)
 Configuração: (q: q2, w: b, Pilha: [Vazia])
 Resultado: REJEITA

Processando: ba

Configuração: (q: q0, w: ba, Pilha: Z)
 Resultado: REJEITA

Processando: vazia

Configuração: (q: q0, w: E, Pilha: Z)

Resultado: REJEITA

Processando: abab

Configuração: (q: q0, w: abab, Pilha: Z)

Configuração: (q: q0, w: bab, Pilha: AZ)

Configuração: (q: q1, w: bab, Pilha: AZ)

Configuração: (q: q1, w: ab, Pilha: Z)

Configuração: (q: q2, w: ab, Pilha: [Vazia])

Resultado: REJEITA

=== Desafio L3: Palíndromos sobre {a,b} ===

Processando: a

Configuração: (q: q0, w: a, Pilha: Z)

Configuração: (q: q1, w: a, Pilha: Z)

Configuração: (q: q2, w: a, Pilha: [Vazia])

Configuração: (q: q0, w: E, Pilha: AZ)

Configuração: (q: q1, w: E, Pilha: AZ)

Configuração: (q: q1, w: E, Pilha: Z)

Configuração: (q: q2, w: E, Pilha: [Vazia])

Resultado: ACEITA

Processando: aba

Configuração: (q: q0, w: aba, Pilha: Z)

Configuração: (q: q1, w: aba, Pilha: Z)

Configuração: (q: q2, w: aba, Pilha: [Vazia])

Configuração: (q: q0, w: ba, Pilha: AZ)

Configuração: (q: q1, w: ba, Pilha: AZ)

Configuração: (q: q0, w: a, Pilha: BAZ)

Configuração: (q: q1, w: a, Pilha: BAZ)

Configuração: (q: q0, w: E, Pilha: ABAZ)

Configuração: (q: q1, w: E, Pilha: ABAZ)

Configuração: (q: q1, w: E, Pilha: BAZ)

Configuração: (q: q1, w: a, Pilha: AZ)

Configuração: (q: q1, w: E, Pilha: Z)

Configuração: (q: q2, w: E, Pilha: [Vazia])

Configuração: (q: q1, w: ba, Pilha: Z)

Configuração: (q: q2, w: ba, Pilha: [Vazia])

Resultado: ACEITA

Processando: abba

Configuração: (q: q0, w: abba, Pilha: Z)
Configuração: (q: q1, w: abba, Pilha: Z)
Configuração: (q: q2, w: abba, Pilha: [Vazia])
Configuração: (q: q0, w: bba, Pilha: AZ)
Configuração: (q: q1, w: bba, Pilha: AZ)
Configuração: (q: q0, w: ba, Pilha: BAZ)
Configuração: (q: q1, w: ba, Pilha: BAZ)
Configuração: (q: q1, w: a, Pilha: AZ)
Configuração: (q: q1, w: E, Pilha: Z)
Configuração: (q: q2, w: E, Pilha: [Vazia])
Configuração: (q: q1, w: ba, Pilha: AZ)
Configuração: (q: q1, w: bba, Pilha: Z)
Configuração: (q: q2, w: bba, Pilha: [Vazia])

Resultado: ACEITA

Processando: ab

Configuração: (q: q0, w: ab, Pilha: Z)
Configuração: (q: q1, w: ab, Pilha: Z)
Configuração: (q: q2, w: ab, Pilha: [Vazia])
Configuração: (q: q0, w: b, Pilha: AZ)
Configuração: (q: q1, w: b, Pilha: AZ)
Configuração: (q: q0, w: E, Pilha: BAZ)
Configuração: (q: q1, w: E, Pilha: BAZ)
Configuração: (q: q1, w: E, Pilha: AZ)
Configuração: (q: q1, w: b, Pilha: Z)
Configuração: (q: q2, w: b, Pilha: [Vazia])

Resultado: REJEITA

Processando: aab

Configuração: (q: q0, w: aab, Pilha: Z)
Configuração: (q: q1, w: aab, Pilha: Z)
Configuração: (q: q2, w: aab, Pilha: [Vazia])
Configuração: (q: q0, w: ab, Pilha: AZ)
Configuração: (q: q1, w: ab, Pilha: AZ)
Configuração: (q: q1, w: b, Pilha: Z)
Configuração: (q: q2, w: b, Pilha: [Vazia])
Configuração: (q: q0, w: b, Pilha: AAZ)
Configuração: (q: q1, w: b, Pilha: AAZ)
Configuração: (q: q0, w: E, Pilha: BAAZ)
Configuração: (q: q1, w: E, Pilha: BAAZ)
Configuração: (q: q1, w: E, Pilha: AAZ)
Configuração: (q: q1, w: b, Pilha: AZ)
Configuração: (q: q1, w: ab, Pilha: Z)
Configuração: (q: q2, w: ab, Pilha: [Vazia])

Resultado: REJEITA

=== Simulador de Máquina de Turing (Parte 3) ===

=== L4: $a^n b^n c^n$ ($n \geq 1$) ===

Processando L4: abc

Fita: ... _ [q0>a] b c ...
Fita: ... X [q1>b] c ...
Fita: ... X Y [q2>c] _ ...
Fita: ... X [q3>Y] Z ...
Fita: ... _ [q3>X] Y Z ...
Fita: ... X [q0>Y] Z ...
Fita: ... X Y [q4>Z] _ ...
Fita: ... X Y Z [q4>_] _ ...
Fita: ... X Y Z _ [q_acc>_] _ ...

Resultado: ACEITA

Processando L4: aabbcc

Fita: ... _ [q0>a] a b b c c ...
Fita: ... X [q1>a] b b c c ...
Fita: ... X a [q1>b] b c c ...
Fita: ... X a Y [q2>b] c c ...
Fita: ... X a Y b [q2>c] c ...
Fita: ... X a Y [q3>b] Z c ...
Fita: ... X a [q3>Y] b Z c ...
Fita: ... X [q3>a] Y b Z c ...
Fita: ... _ [q3>X] a Y b Z c ...
Fita: ... X [q0>a] Y b Z c ...
Fita: ... X X [q1>Y] b Z c ...
Fita: ... X X Y [q1>b] Z c ...
Fita: ... X X Y Y [q2>Z] c ...
Fita: ... X X Y Y Z [q2>c] _ ...
Fita: ... X X Y Y [q3>Z] Z ...
Fita: ... X X Y [q3>Y] Z Z ...
Fita: ... X X [q3>Y] Y Z Z ...
Fita: ... X [q3>X] Y Y Z Z ...
Fita: ... X X [q0>Y] Y Z Z ...
Fita: ... X X Y [q4>Y] Z Z ...
Fita: ... X X Y Y [q4>Z] Z ...
Fita: ... X X Y Y Z [q4>Z] _ ...
Fita: ... X X Y Y Z Z [q4>_] _ ...
Fita: ... X X Y Y Z Z _ [q_acc>_] _ ...

Resultado: ACEITA

Processando L4: aaabbbccc

Fita: ... _ [q0>a] a a b b b c c c ...
Fita: ... X [q1>a] a b b b c c c ...
Fita: ... X a [q1>a] b b b c c c ...
Fita: ... X a a [q1>b] b b c c c ...
Fita: ... X a a Y [q2>b] b c c c ...
Fita: ... X a a Y b [q2>b] c c c ...
Fita: ... X a a Y b b [q2>c] c c ...
Fita: ... X a a Y b [q3>b] Z c c ...
Fita: ... X a a Y [q3>b] b Z c c ...
Fita: ... X a a [q3>Y] b b Z c c ...
Fita: ... X a [q3>a] Y b b Z c c ...
Fita: ... X [q3>a] a Y b b Z c c ...
Fita: ... _ [q3>X] a a Y b b Z c c ...
Fita: ... X [q0>a] a Y b b Z c c ...
Fita: ... X X [q1>a] Y b b Z c c ...
Fita: ... X X a [q1>Y] b b Z c c ...
Fita: ... X X a Y [q1>b] b Z c c ...
Fita: ... X X a Y Y [q2>b] Z c c ...
Fita: ... X X a Y Y b [q2>Z] c c ...
Fita: ... X X a Y Y b Z [q2>c] c ...
Fita: ... X X a Y Y b [q3>Z] Z c ...
Fita: ... X X a Y Y [q3>b] Z Z c ...
Fita: ... X X a Y [q3>Y] b Z Z c ...
Fita: ... X X a [q3>Y] Y b Z Z c ...
Fita: ... X X [q3>a] Y Y b Z Z c ...
Fita: ... X [q3>X] a Y Y b Z Z c ...
Fita: ... X X [q0>a] Y Y b Z Z c ...
Fita: ... X X X [q1>Y] Y b Z Z c ...
Fita: ... X X X Y [q1>Y] b Z Z c ...
Fita: ... X X X Y Y [q1>b] Z Z c ...
Fita: ... X X X Y Y Y [q2>Z] Z c ...
Fita: ... X X X Y Y Y Z [q2>Z] c ...
Fita: ... X X X Y Y Y Z Z [q2>c] _ ...
Fita: ... X X X Y Y Y Z [q3>Z] Z ...
Fita: ... X X X Y Y Y [q3>Z] Z Z ...
Fita: ... X X X Y Y [q3>Y] Z Z Z ...
Fita: ... X X X Y [q3>Y] Y Z Z Z ...
Fita: ... X X X [q3>Y] Y Y Z Z Z ...
Fita: ... X X [q3>X] Y Y Y Z Z Z ...
Fita: ... X X X [q0>Y] Y Y Z Z Z ...
Fita: ... X X X Y [q4>Y] Y Z Z Z ...
Fita: ... X X X Y Y [q4>Y] Z Z Z ...
Fita: ... X X X Y Y Y [q4>Z] Z Z ...
Fita: ... X X X Y Y Y Z [q4>Z] Z ...
Fita: ... X X X Y Y Y Z Z [q4>Z] _ ...

```

Fita: ... X X X Y Y Y Z Z Z [q4>_] _ ...
Fita: ... X X X Y Y Y Z Z Z _ [q_acc>_] _ ...
Resultado: ACEITA

```

Processando L4: aabbc

```

Fita: ... _ [q0>a] a b b c ...
Fita: ... X [q1>a] b b c ...
Fita: ... X a [q1>b] b c ...
Fita: ... X a Y [q2>b] c ...
Fita: ... X a Y b [q2>c] _ ...
Fita: ... X a Y [q3>b] Z ...
Fita: ... X a [q3>Y] b Z ...
Fita: ... X [q3>a] Y b Z ...
Fita: ... _ [q3>X] a Y b Z ...
Fita: ... X [q0>a] Y b Z ...
Fita: ... X X [q1>Y] b Z ...
Fita: ... X X Y [q1>b] Z ...
Fita: ... X X Y Y [q2>Z] _ ...
Fita: ... X X Y Y Z [q2>_] _ ...
Resultado: REJEITA

```

Processando L4: ab

```

Fita: ... _ [q0>a] b ...
Fita: ... X [q1>b] _ ...
Fita: ... X Y [q2>_] _ ...
Resultado: REJEITA

```

Processando L4: abc abc

```

Fita: ... _ [q0>a] b c a b c ...
Fita: ... X [q1>b] c a b c ...
Fita: ... X Y [q2>c] a b c ...
Fita: ... X [q3>Y] Z a b c ...
Fita: ... _ [q3>X] Y Z a b c ...
Fita: ... X [q0>Y] Z a b c ...
Fita: ... X Y [q4>Z] a b c ...
Fita: ... X Y Z [q4>] a b c ...
Resultado: REJEITA

```

Processando L4: vazia

```

Fita: ... _ [q0>_] _ ...
Resultado: REJEITA

```

=== Desafio: Computando $f(n) = n + 1$ (Adição Unária) ===

Processando $f(n)$ para $n = 1$ (Cadeia: 1)

Fita: ... _ [q0>1] _ ...

Fita: ... 1 [q0>_] _ ...

Fita: ... 1 1 [q_acc>_] _ ...

Resultado computado com sucesso!

Processando $f(n)$ para $n = 2$ (Cadeia: 11)

Fita: ... _ [q0>1] 1 ...

Fita: ... 1 [q0>1] _ ...

Fita: ... 1 1 [q0>_] _ ...

Fita: ... 1 1 1 [q_acc>_] _ ...

Resultado computado com sucesso!

Processando $f(n)$ para $n = 4$ (Cadeia: 1111)

Fita: ... _ [q0>1] 1 1 1 ...

Fita: ... 1 [q0>1] 1 1 ...

Fita: ... 1 1 [q0>1] 1 ...

Fita: ... 1 1 1 [q0>1] _ ...

Fita: ... 1 1 1 1 [q0>_] _ ...

Fita: ... 1 1 1 1 1 [q_acc>_] _ ...

Resultado computado com sucesso!

Tabela 1: Resultados dos Testes Processados no Sistema

Máquina	Linguagem	Cadeia Processada	Resultado Sistema
AFD	L_1 (Termina em ab)	$ababab$	ACEITA
AFD	L_1 (Termina em ab)	ba	REJEITA
AP	$L_2 = a^n b^n$	$aabb$	ACEITA
AP	$L_2 = a^n b^n$	abb	REJEITA
AP	L_3 (Palíndromo)	$abba$	ACEITA
MT	$L_4 = a^n b^n c^n$	$aaabbbccc$	ACEITA
MT	Adição Unária $f(n)$	$1111 \rightarrow 11111$	ACEITA (Parada)

6 Análise Comparativa de Poder Computacional

O desenvolvimento consecutivo destas três máquinas permitiu uma análise clara e prática da Hierarquia de Chomsky.

Os **Autômatos Finitos** (AFDs) demonstraram possuir o poder computacional mais restrito. Por não possuírem memória auxiliar, sua aplicação limita-se a reconhecer padrões sequenciais

e repetições (como L_1). Eles não conseguem, por exemplo, contar ou balancear símbolos. A tentativa de processar $a^n b^n$ com um AFD exigiria um número infinito de estados, o que viola a sua definição.

Os **Autômatos de Pilha** (APs) resolvem o problema da contagem ao adicionar a estrutura de pilha (memória finita em profundidade). Com isso, tornam-se capazes de reconhecer pares balanceados, como casar os 'a's com os 'b's (L_2). Contudo, a limitação de acesso estrito ao topo da pilha impede que o AP reconheça padrões complexos de cruzamento tridimensional, tornando-o incapaz de lidar com linguagens como $a^n b^n c^n$.

As **Máquinas de Turing** (MT) demonstraram o ápice do poder computacional. Com a introdução de uma fita de leitura/escrita infinita e um cabeçote móvel para a esquerda e direita, a MT atinge o conceito de completude de Turing. Esta máquina não apenas consegue memorizar e contar, mas também remarcar dados lidos anteriormente (essencial para reconhecer L_4) e realizar cálculos numéricos absolutos, como comprovado pela execução da máquina de adição unária.

7 Conclusão

A execução deste projeto proporcionou uma solidificação inestimável dos conceitos abordados na disciplina. Traduzir o jargão matemático rigoroso (como 5-tuplas e relações de transição) para código limpo e funcional em C# quebrou a abstração teórica, provando que matrizes, dicionários e pilhas do cotidiano de desenvolvimento de software são, no fundo, modelos matemáticos.

Compreender o porquê de um JSON Parser ou Compilador precisar de uma árvore estruturada em vez de um simples RegEx (pois RegEx não tem memória de pilha) muda a forma de arquitetar softwares no mercado de trabalho. A teoria da computação deixa de ser uma disciplina isolada e consolida-se como a base irrefutável de qualquer processo de engenharia de software moderno.

Referências

- [1] MENEZES, P. B. *Linguagens formais e autômatos*. 6. ed. Porto Alegre: Bookman, 2010.
- [2] HOPCROFT, J. E.; MOTWANI, R.; ULLMAN, J. D. *Introdução à teoria de autômatos, linguagens e computação*. Rio de Janeiro: Elsevier, 2002.
- [3] SIPSER, M. *Introdução à teoria da computação*. São Paulo: Cengage Learning, 2007.